ATmega32 Embedded Systems Architecture

LED Control, Temperature Sensing, and LCD Display Integration

Ahmed Abdelkader (ID: 2004439)
German International University (GIU), Faculty of Engineering
CSEN701: Embedded Systems (Spring 2022)
Supervisor: Dr. Eng. Amr T. Abdel-Hamid
Repository: https://github.com/ahmed88ii/GIU-2601

Executive Summary

This technical report details the architecture and implementation of an embedded system utilizing the ATmega32 microcontroller. The project integrates multiple core peripherals into a cohesive, interrupt-driven application. Key features include software-debounced physical inputs, analog temperature acquisition via an LM35 sensor, real-time data visualization on an HD44780 LCD, and Timer1 CTC interrupts for non-blocking scheduling. The primary objective was to architect a robust firmware environment that avoids busy-waiting and maintains a highly responsive main execution loop.

1. System Architecture

The system centers on the ATmega32 microcontroller operating at an 8 MHz clock frequency. It interfaces with physical user inputs and thermal sensors while maintaining strict timing constraints utilizing an internal Timer1 ISR.

1.1. Technical Specifications

Parameter	Specification
Microcontroller	ATmega32 @ 8 MHz
User Input	3 Push-buttons (software debounced)
Status Indicator	1 LED toggled at 500 ms intervals via Timer1
Thermal Sensor	LM35 on ADC0 (Sample rate: 1.0 Hz)
Display Component	16×2 HD44780 LCD (4-bit mode)
Toolchain	`avr-gcc`, `avrdude`, USBasp

2. Hardware Design and Calibration

2.1. I/O Mapping

- **Inputs (PB0–PB2):** Configured with external 10kΩ pull-up resistors; active LOW logic.
- **Outputs (PD0–PD3):** Drive standard LEDs via 330Ω current-limiting resistors.
- **Analog (PA0):** Interfaces with the LM35 (AVCC = 5 V, AREF = 5 V).
- **Display (PC0–PC5):** Manages LCD RS, EN, and Data lines (D4–D7).

2.2. Sensor Calibration

The LM35 thermal sensor provides a linear output of 10 mV/°C. Operating with a 5 V reference, the 10-bit ADC yields the following calculation:

$$T\ (^\circ\mathrm{C}) \ = \ \frac{\mathrm{ADC} \times 500}{1024} \tag{1}$$

To optimize memory footprint and CPU cycles, the result is stored in tenths of a degree (e.g., `237` represents `23.7 °C`), strictly avoiding floating-point arithmetic.

3. Software Architecture

The codebase is highly modularized, strictly enforcing the separation of concerns.

3.1. Core Design Principles

1. **Flag-Driven Interrupts:** The Timer1 ISR exclusively sets volatile execution flags (`g_blink_tick`, `g_temp_tick`). It never directly calls blocking operations like I/O or ADC conversions. The main loop polls and clears these flags, guaranteeing minimal ISR execution time.
2. **State-Machine Debouncing:** The `Button_WasPressed()` routine validates pin states across a 20 ms temporal window, ensuring exactly one clean logic edge per physical actuation.

4. Firmware Implementation

4.1. Main Execution Super-Loop

The primary routine manages peripheral states based on background timing flags.

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include "led.h"
#include "button.h"
#include "adc.h"
```

```c
#include "lcd.h"
#include "timer.h"

int main(void) {
    LED_Init();
    Button_Init();
    ADC_Init();
    LCD_Init();
    Timer1_Init();
    sei();  // Enable global interrupts

    LCD_SetCursor(0, 0);
    LCD_Print("Temp:");

    while (1) {
        if (g_blink_tick) {
            g_blink_tick = 0;
            LED_Toggle(BLINK_LED);
        }

        if (g_temp_tick) {
            g_temp_tick = 0;
            uint16_t t = ADC_ReadLM35();
            char buf[6];
            buf[0] = '0' + (t / 100);
            buf[1] = '0' + ((t / 10) % 10);
            buf[2] = '.';
            buf[3] = '0' + (t % 10);
            buf[4] = 'C';
            buf[5] = '\0';
            LCD_SetCursor(0, 6);
            LCD_Print(buf);
        }

        for (uint8_t i = 0; i < 3; i++) {
            if (Button_WasPressed(i)) {
                LED_Toggle(i);
            }
        }
    }
}
```

4.2. Timer1 CTC Interrupt Service Routine

Generates reliable scheduling ticks without stalling the main execution flow.

```c
#include <avr/io.h>
#include <avr/interrupt.h>
#include "timer.h"
```

```

volatile uint8_t g_blink_tick = 0;
volatile uint8_t g_temp_tick  = 0;
static uint8_t blink_count = 0;

void Timer1_Init(void) {
    // CTC mode, prescaler = 256
    // 8 MHz / 256 = 31250 ticks/sec
    // OCR1A = 15624 => 0.5 s interrupt
    TCCR1B |= (1 << WGM12) | (1 << CS12);
    OCR1A   = 15624;
    TIMSK  |= (1 << OCIE1A);
}

ISR(TIMER1_COMPA_vect) {
    g_blink_tick = 1;
    blink_count++;
    if (blink_count >= 2) {
        blink_count = 0;
        g_temp_tick = 1;
    }
}
```

5. Validation and System Challenges

- **Input Reliability:** The 20 ms software debounce window completely eliminated ghost inputs and mechanical bouncing artifacts during rapid actuation cycles.
- **LCD Initialization Sequence:** Strict adherence to HD44780 power-on timing constraints (15 ms, 4.1 ms, 100 μs delays) was required to prevent character corruption upon initial boot.
- **ISR Discipline:** Abstracting I/O out of the Timer1 ISR into the main loop via boolean flags successfully mitigated intermittent display tearing caused by prolonged interrupt blocking.

6. Conclusion

This deployment successfully validates the integration of multiple peripheral subsystems on the ATmega32 architecture. By strictly adhering to embedded design principles—minimizing ISR overhead and orchestrating I/O via a non-blocking super-loop—the firmware achieves high operational stability and scalability.